

MAVIS --- Memory Augmented Visual Incremental System Complete Technical Project Specification

Problem Statement: Memory-Augmented Neural Network for Incremental Learning of New Visual Categories

Project type: Deep Learning / Continual Learning / Neural Memory / Attention Primary dataset: Tiny ImageNet Framework: TensorFlow + Keras GPU target: NVIDIA RTX 4050 6 GB Training constraint: No pretrained models or pretrained feature extractors Core models: CNN baseline, CNN+EWC, CNN+Attention Memory, MAVIS

Primary research question:

Can an adaptive external neural memory, retrieved through attention and combined with parameter-importance regularization, reduce catastrophic forgetting when a network incrementally learns new visual categories under a fixed memory budget?

1. Project Overview

MAVIS is a memory-augmented neural network designed to address the problem of incremental learning of new visual categories: it must learn new visual classes as they arrive, one group at a time, while retaining useful knowledge about the visual categories it has already learned.

The system is intentionally built from scratch. The convolutional encoder, classifier, EWC mechanism, external memory, attention mechanism, memory-update policy, and fusion mechanism are implemented by the project team.

The project studies two complementary forms of knowledge retention:

1. Parameter-level retention: EWC protects important neural-network parameters from large changes.
2. Representation-level retention: an external memory stores selected visual feature representations from previous tasks.

Attention provides selective retrieval from the external memory, while adaptive memory management decides which representations remain in the fixed memory budget.

The final system is:

Tiny ImageNet | v Continual Task Split | v CNN Encoder (from scratch) | 512/1024-D feature /
/
v v EWC module Query Network | v External Neural Memory | v Multi-Head Attention | v Retrieved Feature |
Current Feature + Memory Feature | v Gated Fusion | v Classifier | v Prediction | v Adaptive Memory Update
| v Next Task

2. Research Objective

2.1 Main objective Develop and evaluate a memory-augmented continual-learning architecture that can learn new visual classes sequentially without severe degradation on previously learned classes.

2.2 Secondary objectives

- Quantify catastrophic forgetting in a standard CNN.
- Evaluate EWC as a parameter-protection mechanism.
- Evaluate external neural memory independently.
- Evaluate attention-based memory retrieval.
- Combine EWC and attention memory into MAVIS.
- Study performance under fixed memory budgets.
- Study the effect of memory size and attention configuration.
- Compare accuracy, forgetting, compute cost, and memory cost.
- Determine whether the proposed components complement each other.

2.3 What the project does NOT claim The project should not claim:

- zero catastrophic forgetting,
- perfect classification,
- universally optimal architecture,
- superiority on all datasets,
- or guaranteed improvement before experiments are performed.

All such conclusions must be based on measured experimental results.

3. Dataset Strategy

3.1 Primary dataset: Tiny ImageNet Tiny ImageNet is the main research dataset because it provides substantially more classes and images than CIFAR-100 while remaining manageable on a 6 GB GPU.

Target configuration: Classes: 200 Image resolution: 64 x 64 Channels: 3 RGB Training images: approximately 100,000 Validation images: approximately 10,000

The exact local dataset statistics must be verified after download.

3.2 Development dataset: CIFAR-100 CIFAR-100 should be used during development because it is much faster to train.

Purpose:

- validate the data pipeline,
- validate the continual-task generator,
- debug the CNN,
- debug EWC,
- debug memory retrieval,
- debug checkpoints,
- perform fast ablations.

CIFAR-100 is not required to be the primary final dataset.

3.3 No artificial duplication Do not increase dataset size by duplicating identical images.

Dataset diversity should come from:

- the original dataset,
- legitimate augmentation,
- and, if available within the project scope, an additional independent dataset.

4. Continual-Learning Task Definition

Tiny ImageNet contains 200 classes. The main experiment will use:

10 tasks x 20 classes

Example: Task 1: classes 0-19 Task 2: classes 20-39 Task 3: classes 40-59 Task 4: classes 60-79 Task 5: classes 80-99 Task 6: classes 100-119 Task 7: classes 120-139 Task 8: classes 140-159 Task 9: classes 160-179 Task 10: classes 180-199

The class order must be fixed and recorded in the experiment configuration.

A second task ordering can optionally be generated with a fixed random seed to test sensitivity to class order.

4.1 Alternative task granularity After the primary experiment works, optional experiments can test:

5 tasks x 40 classes 10 tasks x 20 classes 20 tasks x 10 classes 40 tasks x 5 classes

This determines whether forgetting changes as the number of incremental stages increases.

5. Data Pipeline

5.1 Loading The pipeline must:

1. Locate Tiny ImageNet files.
2. Parse class IDs.
3. Map class IDs to integer labels.
4. Load RGB images.
5. Resize/crop only if required.
6. Normalize pixel values.
7. Apply training augmentation.
8. Construct task-specific datasets.
9. Batch the data.
10. Prefetch batches.

5.2 Recommended TensorFlow pipeline Conceptually:

```
dataset = tf.data.Dataset.from_tensor_slices((images, labels)) dataset = dataset.shuffle(
buffer_size=SHUFFLE_BUFFER, seed=SEED ) dataset = dataset.map( augmentation_function,
num_parallel_calls=tf.data.AUTOTUNE ) dataset = dataset.batch(BATCH_SIZE) dataset =
dataset.prefetch(tf.data.AUTOTUNE)
```

The final implementation should avoid loading unnecessary copies of the complete dataset into GPU memory.

5.3 Augmentation Initial augmentation:

- random horizontal flip,
- random crop/padding,
- moderate color perturbation.

Optional:

- random erasing,
- MixUp,
- CutMix.

For the main scientific comparison, augmentation must be identical across the four core models.

6. Hardware and Runtime Configuration

Target hardware: GPU: NVIDIA RTX 4050 VRAM: 6 GB

Recommended initial settings: Framework: TensorFlow/Keras Batch size: 64 Image size: 64 x 64 Mixed precision: FP16 where supported Workers/pipeline: tf.data AUTOTUNE

The exact batch size must be determined experimentally based on GPU memory usage.

6.1 GPU verification Before training:

```
import tensorflow as tf print(tf.config.list_physical_devices("GPU"))
```

The expected result should contain a GPU device.

Also record:

- TensorFlow version,
- CUDA/cuDNN environment,
- NVIDIA driver version,
- Python version,
- GPU model,
- GPU VRAM.

These should be saved in the experiment metadata.

7. Software Architecture

Recommended directory:

```
MAVIS/ ├── README.md ├── requirements.txt ├── config.py ├── main.py | ├── data/ | ├── init.py |  
| ├── download.py | ├── tiny_imagenet.py | ├── task_generator.py | └── augmentation.py | ├──  
models/ | ├── init.py | ├── encoder.py | ├── classifier.py | ├── attention.py | ├── memory.py | ├──  
fusion.py | └── mavis.py | ├── continual/ | ├── init.py | ├── ewc.py | ├── trainer.py | ├──  
task_manager.py | └── checkpoint.py | ├── experiments/ | ├── baseline_cnn.py | ├──  
ewc_experiment.py | ├── attention_memory.py | ├── mavis_experiment.py | └── ablation.py | ├──  
evaluation/ | ├── accuracy.py | ├── forgetting.py | ├── metrics.py | ├── confusion_matrix.py | └──  
plots.py | ├── configs/ | ├── baseline.yaml | ├── ewc.yaml | ├── memory.yaml | └── mavis.yaml |  
| ├── checkpoints/ | ├── baseline/ | ├── ewc/ | ├── memory/ | └── mavis/ | ├── results/ | ├── raw/  
| ├── metrics/ | ├── figures/ | └── tables/ | └── notebooks/ | ├── dataset_analysis.ipynb ├──  
training_analysis.ipynb └── results_analysis.ipynb
```

8. Model 1 --- CNN Baseline

The first model is a conventional CNN trained sequentially.

8.1 Purpose The baseline establishes the amount of catastrophic forgetting produced without a continual-learning mechanism.

8.2 Proposed encoder Initial architecture:

Input: 64x64x3 Conv2D 64, 3x3 BatchNorm ReLU Conv2D 64, 3x3 BatchNorm ReLU MaxPool Conv2D 128, 3x3 BatchNorm ReLU Conv2D 128, 3x3 BatchNorm ReLU MaxPool Conv2D 256, 3x3 BatchNorm ReLU Conv2D 256, 3x3 BatchNorm ReLU MaxPool Conv2D 512, 3x3 BatchNorm ReLU Global Average Pooling Dense -> 512

The exact channel counts may be reduced if GPU memory becomes a bottleneck.

8.3 Classifier

512-D feature | Dense 256 | ReLU | Dropout (optional) | Dense 200 | Softmax

For class-incremental training, the classifier may be implemented as a fixed 200-class output layer while masking unavailable classes during task-specific training, or as an incremental classifier. The choice must remain consistent across all core models.

9. Phase 1 --- Environment Setup

Step 1. Install Python Use a project-specific virtual environment.

Step 2. Install TensorFlow Install a TensorFlow version compatible with the selected CUDA environment.

Step 3. Verify GPU Run: `import tensorflow as tf print(tf.config.list_physical_devices("GPU"))`

Step 4. Verify numerical computation Run a GPU matrix multiplication.

Step 5. Record environment Save: Python version TensorFlow version CUDA version cuDNN version
NVIDIA driver GPU model

Deliverable A reproducible environment in which TensorFlow successfully executes GPU workloads.

10. Phase 2 --- Dataset Preparation

Step 1 Download Tiny ImageNet.

Step 2 Build a class-to-index mapping.

Step 3 Verify:

- image count,
- class count,
- image dimensions,
- corrupted images,
- missing images.

Step 4 Create training/validation/test structures as appropriate.

Step 5 Create the continual task generator.

Step 6 Save task definitions.

Example: { "task_1": [0, 1, 2, "...", 19], "task_2": [20, 21, 22, "...", 39] }

Deliverable A deterministic task-generation pipeline.

11. Phase 3 --- Data Augmentation and Input Pipeline

Implement: Random crop Random horizontal flip Normalization Batching Prefetching

Optional: Color jitter Random erasing MixUp CutMix

Measure input throughput. The GPU should not spend most of its time waiting for CPU data loading.

Deliverable A benchmarked TensorFlow tf.data pipeline.

12. Phase 4 --- CNN Baseline Training

Train the baseline sequentially.

For each task: Load task dataset ↓ Train CNN ↓ Evaluate current task ↓ Evaluate all previous tasks ↓ Save checkpoint ↓ Move to next task

At the end of Task 10, evaluate all 200 classes.

Required outputs For each task:

- training loss,
- training accuracy,
- current-task validation accuracy,
- previous-task accuracy,
- total evaluation accuracy,
- training time,
- GPU memory.

Deliverable The first complete continual-learning accuracy matrix.

13. Phase 5 --- Catastrophic Forgetting Measurement

Let $a_{\{t,i\}}$ be the accuracy on task i after training task t .

Construct: Evaluation Task T1 T2 T3 ... T10 After T1 ✓ After T2 ✓ ✓ After T3 ✓ ✓ ✓ ... After T10 ✓ ✓ ✓ ...
✓

This is the task accuracy matrix.

Forgetting For task i : $F_i = \max_{\{t \in \{1, \dots, T\}\}} a_{\{t,i\}} - a_{\{T,i\}}$

Average forgetting: $F_{\text{avg}} = (1/(T-1)) * \sum_{i=1}^{T-1} F_i$

The baseline gives us the reference forgetting level.

14. Phase 6 --- Implement EWC

EWC protects important parameters.

14.1 Save old parameters After Task t , save: $\theta^* = \theta_t$ for trainable parameters.

14.2 Estimate Fisher information A practical diagonal Fisher approximation is: $F_i \approx (1/N) * \sum_{n=1}^N (d(\log p(y_n|x_n, \theta)) / d(\theta_i))^2$

Use a subset of training samples to control computation.

Suggested initial value: 500-2000 samples/task

14.3 EWC loss During the next task: $L_{\text{total}} = L_{\text{CE}} + (\lambda/2) * \sum_i F_i (\theta_i - \theta_i^*)^2$

where:

- L_{CE} = classification cross-entropy,
- F_i = parameter importance,
- θ_i^* = old parameter,
- θ_i = current parameter,
- λ = EWC strength.

14.4 Important implementation detail The Fisher values and old parameters must not receive gradients. They are reference tensors.

Deliverable CNN+EWC model with measurable forgetting.

15. Phase 7 --- EWC Experiment

Run: CNN vs CNN + EWC

Keep everything else identical.

Compare:

- average accuracy,
- final accuracy,
- forgetting,
- training time,
- GPU memory.

Perform a small EWC lambda sweep: $\lambda = 10$ $\lambda = 100$ $\lambda = 1000$ $\lambda = 5000$

The exact useful range must be determined empirically. Do not assume a particular lambda is optimal.

16. Phase 8 --- External Neural Memory

The memory stores compact visual representations rather than raw images.

Each entry can contain: feature vector class label importance score optional task ID

Example: MemoryEntry: feature: 512-D vector label: integer importance: float task_id: integer

Memory budget Test fixed capacities: 50 100 250 500 1000

This allows direct analysis of the memory-performance trade-off.

17. Phase 9 --- Memory Storage

After processing training samples, generate: $f = \text{Encoder}(x)$

The feature f becomes a memory candidate.

The system must not store every feature. Otherwise the memory grows without bound.

Instead: Candidate feature $\downarrow$ Importance / novelty calculation $\downarrow$ Memory has free slot? | yes | no | $\vee$ Store
Replace least useful candidate

18. Phase 10 --- Adaptive Memory Selection

The first implementation can use a simple importance function.

Possible components:

Novelty Measure distance from existing memory: $N(x) = \min_j \|f_x - m_j\|_2$

A high value means the feature is not well represented by existing memory.

Diversity Encourage memory entries that cover different regions of feature space.

Class balance Prevent the memory from becoming dominated by a small number of classes.

Importance The final score can be defined as a weighted combination: $S_i = \alpha N_i + \beta D_i + \gamma C_i$

The exact formulation should be treated as a research component and experimentally justified.

19. Phase 11 --- Attention Memory

Attention determines which memory entries are relevant to the current feature.

Query $Q = W_Q f$

Keys $K = W_K M$

Values $V = W_V M$

where M represents stored memory features.

Scaled dot-product attention $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$

The result is a retrieved representation: $r = \text{Attention}(Q, K, V)$

Multi-head attention Initial configuration: Heads: 4 Query dimension: 256 Key dimension: 256 Value dimension: 512

These are starting values, not guaranteed optimal values.

20. Phase 12 --- Memory Retrieval

For every incoming image: Image $\downarrow$ CNN $\downarrow$ Current feature $\downarrow$ Query projection $\downarrow$ Attention over memory $\downarrow$ Weighted memory retrieval $\downarrow$ Retrieved feature

The attention matrix provides a probability distribution over memory entries.

Example: Memory entry 1 $\rightarrow$ 0.01 Memory entry 2 $\rightarrow$ 0.02 Memory entry 3 $\rightarrow$ 0.41 Memory entry 4 $\rightarrow$ 0.31
...

The weighted combination becomes the retrieved memory representation.

21. Phase 13 --- Gated Feature Fusion

The current representation and retrieved memory representation must be combined.

Let: f_c = current feature f_m = retrieved memory

Concatenate: $z = [f_c; f_m]$

Compute a gate: $g = \text{sigmoid}(W_g z + b_g)$

Then: $f_{\text{final}} = g \odot f_c + (1-g) \odot f_m$

Interpretation:

- $g \approx 1$: rely more on current knowledge.
- $g \approx 0$: rely more on retrieved knowledge.

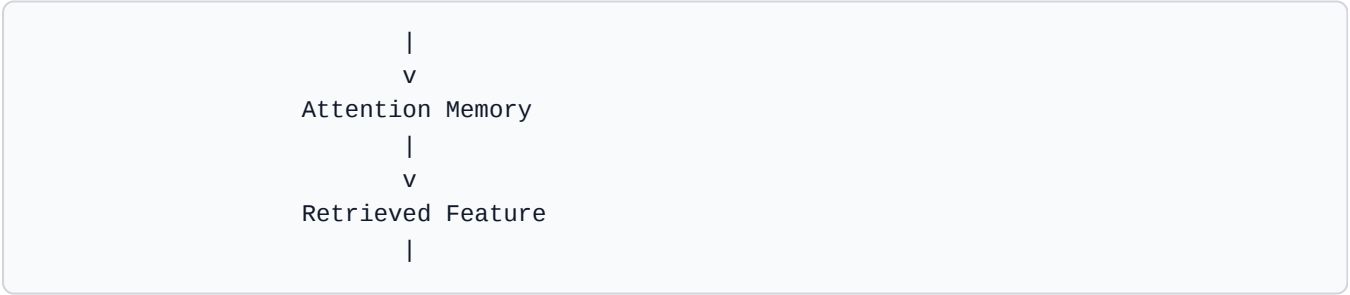
This allows the network to adapt the amount of memory influence.

22. Phase 14 --- MAVIS Architecture

The final proposed model contains:

Input | CNN Encoder |

Current feature
$v \ v$
EWC Query Network



Current Feature + Retrieved Feature | v Gated Fusion | v Classifier | v Prediction | v Adaptive Memory Update

EWC operates on the trainable parameters. Memory operates on feature representations.

These mechanisms are therefore complementary rather than identical.

23. Phase 15 --- MAVIS Training Procedure

For each continual task:

Step 1 Load the current task.

Step 2 Train the CNN and MAVIS components.

Step 3 Compute classification loss.

Step 4 Compute EWC penalty if previous-task knowledge exists.

Step 5 Compute: $L_{\text{total}} = L_{\text{classification}} + \lambda * L_{\text{EWC}}$

Step 6 Backpropagate.

Step 7 Update trainable parameters.

Step 8 Generate feature representations for memory candidates.

Step 9 Score memory candidates.

Step 10 Update the fixed-size external memory.

Step 11 Evaluate current and previous tasks.

Step 12 Save checkpoint.

Step 13 Move to the next task.

24. Phase 16 --- Core Experimental Models

Only four core models are required.

Model 1 --- CNN CNN Purpose: Measure catastrophic forgetting.

Model 2 --- CNN + EWC CNN + EWC Purpose: Measure parameter-level protection.

Model 3 --- CNN + Attention Memory CNN + External Memory + Attention Retrieval Purpose: Measure representation-level memory.

Model 4 --- MAVIS CNN + EWC + Adaptive Memory + Attention + Gated Fusion Purpose: Evaluate the complete proposed architecture.

25. Phase 17 --- Ablation Experiments

Ablation studies determine which components actually matter.

Ablation A Remove EWC: MAVIS - EWC

Ablation B Remove attention: MAVIS - Attention Use a simpler retrieval mechanism.

Ablation C Remove adaptive memory: MAVIS - Adaptive Selection Use a fixed/random replacement strategy.

Ablation D Change memory capacity: 50 100 250 500 1000

Ablation E Change attention heads: 1 2 4 8

Only run these after the core MAVIS pipeline is stable.

26. Phase 18 --- Evaluation Metrics

26.1 Task accuracy Measure accuracy on each task after every subsequent task.

26.2 Average accuracy $A_{avg} = (1/T) * \sum_{i=1}^T a_i$

26.3 Final accuracy Accuracy across all learned classes after Task T.

26.4 Forgetting $F_i = \max_t a_{\{t,i\}} - a_{\{T,i\}}$

26.5 Average forgetting $F_{avg} = (1/(T-1)) * \sum_{i=1}^{T-1} F_i$

26.6 Old-class accuracy Evaluate only classes learned before the current task.

26.7 New-class accuracy Evaluate classes introduced by the current task.

26.8 Memory efficiency Record: number of memory entries bytes per entry total memory size GPU memory

26.9 Compute efficiency Record: training time time/task time/epoch inference latency parameter count

27. Phase 19 --- Required Visualizations

Generate at least:

Plot 1 --- Accuracy vs task x = task number y = accuracy lines = four core models

Plot 2 --- Forgetting vs task x = task number y = forgetting

Plot 3 --- Memory budget vs accuracy x = memory entries y = average accuracy

Plot 4 --- Memory budget vs forgetting x = memory entries y = average forgetting

Plot 5 --- Ablation comparison Compare: Full MAVIS MAVIS - EWC MAVIS - Attention MAVIS - Adaptive Memory

Plot 6 --- Task accuracy matrix Display the complete (T x T) accuracy matrix as a heatmap.

28. Phase 20 --- Checkpointing

A checkpoint should be created after every task.

Example: checkpoints/ └─ mavis/ └─ task_01/ └─ task_02/ └─ task_03/ └─ task_04/ └─ task_05/
└─ task_06/ └─ task_07/ └─ task_08/ └─ task_09/ └─ task_10/

A checkpoint must contain enough state to resume training:

- model weights,
- optimizer state,
- current task,
- EWC reference parameters,

- Fisher information,
- memory contents,
- random seeds/configuration.

29. Phase 21 --- Reproducibility

Every experiment must record: random seed dataset version task order task configuration batch size learning rate optimizer epochs augmentation model architecture feature dimension memory size attention heads EWC lambda Fisher sample count TensorFlow version GPU information training time

Use fixed seeds for comparison experiments. Where TensorFlow/GPU nondeterminism exists, document it rather than claiming perfect determinism.

30. Phase 22 --- Configuration Management

Example configuration:

dataset: name: tiny_imagenet image_size: 64 num_classes: 200 classes_per_task: 20 num_tasks: 10
training: batch_size: 64 epochs_per_task: 20 learning_rate: 0.001 optimizer: adam model: feature_dim: 512
hidden_dim: 256 memory: capacity: 500 strategy: adaptive attention: heads: 4 key_dim: 256 ewc: enabled:
true lambda: 1000 fisher_samples: 1000

The actual final values must be selected through controlled experiments.

31. Phase 23 --- Training-Time Strategy

Do not immediately train the full Tiny ImageNet experiment for many epochs.

Use the following progression.

Stage A --- Unit testing Use tiny subsets. Goal: Does every component execute?

Stage B --- CIFAR-100 prototype Goal: Does continual learning work?

Stage C --- Tiny ImageNet smoke test Use: 1-2 tasks, 1-3 epochs Goal: Does the complete pipeline execute on RTX 4050?

Stage D --- Full Tiny ImageNet baseline Train the four core models.

Stage E --- Ablations Only after the main results are stable.

32. Phase 24 --- Approximate Training Budget

Actual time must be measured on the user's machine.

A rough planning estimate for Tiny ImageNet is: One epoch: approximately tens of seconds, depending on architecture, batch size, data pipeline and GPU utilization.

For 10 tasks x 20 epochs: 200 task-epochs

If one epoch averages 30-45 seconds: $200 \times 30 \text{ sec} = 6,000 \text{ sec} \approx 1.7 \text{ hours}$ $200 \times 45 \text{ sec} = 9,000 \text{ sec} \approx 2.5 \text{ hours}$

A more complex MAVIS implementation can take longer because attention and memory retrieval add computation.

Therefore, the first real benchmark should be measured rather than assumed.

33. Phase 25 --- Memory Complexity

If memory contains M entries and the key dimension is d , attention requires approximately $O(Md)$ for the query-key similarity per sample.

With: $M = 500$ $d = 256$

the attention operation is small compared with the CNN's overall computation.

At very large memory sizes, retrieval cost becomes increasingly important.

This is why fixed-memory experiments are valuable.

34. Phase 26 --- GPU Memory Management

The RTX 4050 has 6 GB VRAM.

Use:

- batch-size tuning,
- mixed precision,
- efficient tf.data,
- feature dimension control,
- checkpointing,
- memory-efficient attention,
- gradient accumulation only if required.

If an out-of-memory error occurs:

1. Reduce batch size.
2. Reduce feature dimension.
3. Reduce model channels.
4. Reduce attention dimensions.
5. Enable/verify mixed precision.
6. Check whether unnecessary tensors are retained.

Do not immediately shrink the research idea; first identify the actual memory bottleneck.

35. Phase 27 --- Research Questions

The final report should answer explicit questions.

RQ1 How severe is catastrophic forgetting in a CNN trained sequentially?

RQ2 Does EWC reduce forgetting?

RQ3 Does external attention-based memory reduce forgetting?

RQ4 Does combining EWC and attention memory provide complementary benefits?

RQ5 How does memory capacity affect accuracy and forgetting?

RQ6 How does the number of continual-learning tasks affect performance?

RQ7 How much additional computation does MAVIS introduce?

36. Phase 28 --- Hypotheses

These are hypotheses, not conclusions.

H1 Sequential CNN training will exhibit measurable catastrophic forgetting.

H2 EWC will reduce parameter drift and may reduce forgetting.

H3 External memory may preserve useful visual representations.

H4 Attention may improve selective retrieval compared with non-selective retrieval.

H5 Combining parameter protection and external memory may provide complementary retention.

H6 Increasing memory capacity may improve retention up to a point, after which returns may diminish.

Experiments determine whether these hypotheses are supported.

37. Phase 29 --- Final Results Table

The final report should include a table similar to:

Model	Avg Acc.	Final Acc.	Avg Forgetting	Memory	Params	Time	CNN	measured	measured	measured	0
measured	measured	measured	CNN + EWC	measured	measured	measured	0	measured	measured	CNN + Attention	
Memory	measured	measured	measured	measured	measured	measured	MAVIS	measured	measured		
measured	measured	measured	measured								

Do not fill the values before running the experiments.

38. Phase 30 --- Failure Analysis

If MAVIS does not outperform a baseline, that is still a valid experimental result.

Investigate:

- attention weights,
- memory class distribution,
- memory diversity,
- EWC penalty magnitude,
- classifier bias,
- task order,
- memory capacity,
- feature quality,
- gradient magnitudes,
- training stability.

Potential failure modes:

Failure 1 --- Memory dominated by a few classes Solution: Add class-balanced memory constraints.

Failure 2 --- Attention becomes nearly uniform Possible causes:

- poor feature representation,
- unsuitable query/key projections,
- insufficient training,
- scaling problem.

Failure 3 --- EWC prevents new learning Possible cause: lambda is too large. Run a lambda sweep.

Failure 4 --- Memory has little effect Possible causes:

- poor stored representations,
- memory too small,
- retrieval not selective,
- classifier ignores retrieved features.

Failure 5 --- GPU out of memory Reduce: batch size feature dimension attention dimension model width

39. Phase 31 --- Code Quality Requirements

The implementation should:

- avoid duplicated training logic,
- use reusable TensorFlow/Keras components,
- separate data/model/training/evaluation code,
- save configurations,
- save checkpoints,

- log metrics,
- use type hints where useful,
- document non-obvious algorithms,
- use assertions for tensor shapes,
- validate task labels,
- validate memory capacity.

Each major component should have unit tests.

40. Phase 32 --- Unit Tests

Minimum tests:

Dataset Verify: image shape == (64,64,3), label range == [0,199]

Task generator Verify: 10 tasks, 20 classes/task, no unintended class overlap

Encoder Verify output: (batch, feature_dim)

Memory Verify: len(memory) <= capacity

Attention Verify: attention weights sum approximately to 1

EWC Verify: EWC loss == 0 when current parameters equal stored old parameters.

Checkpoint Verify that training can resume from a task checkpoint.

41. Phase 33 --- Team Division

For a four-person team:

Member 1 --- Data + Infrastructure Responsible for:

- Tiny ImageNet pipeline,
- task generator,
- augmentation,
- configuration,
- checkpoint infrastructure.

Member 2 --- CNN + EWC Responsible for:

- encoder,
- classifier,
- baseline training,
- Fisher computation,
- EWC loss.

Member 3 --- Memory + Attention Responsible for:

- memory structure,
- memory selection,
- query/key/value projections,
- attention,
- retrieval.

Member 4 --- Evaluation + Experiments Responsible for:

- metrics,
- forgetting,
- plots,
- experiment runner,
- ablation framework,
- result tables.

All four members should understand the complete architecture before final integration.

42. Phase 34 --- Integration Order

Integration must happen in this order:

Data ↓ CNN ↓ Continual trainer ↓ Evaluation ↓ EWC ↓ External memory ↓ Attention ↓ Fusion ↓ MAVIS ↓ Ablations

Do not integrate all components simultaneously.

43. Phase 35 --- Final Experiment Matrix

The minimum final experiment set:

E1 CNN baseline E2 CNN + EWC E3 CNN + Attention Memory E4 MAVIS E5 MAVIS memory=50 E6 MAVIS memory=100 E7 MAVIS memory=250 E8 MAVIS memory=500 E9 MAVIS memory=1000 E10 MAVIS without EWC E11 MAVIS without Attention E12 MAVIS without Adaptive Memory

Optional: E13 attention heads = 1 E14 attention heads = 2 E15 attention heads = 4 E16 attention heads = 8

Do not run every optional experiment unless time permits.

44. Phase 36 --- Final Project Workflow

The complete project is:

PHASE 1 Environment ↓ PHASE 2 Dataset ↓ PHASE 3 Task Generator ↓ PHASE 4 CNN Baseline ↓ PHASE 5 Forgetting Evaluation ↓ PHASE 6 EWC ↓ PHASE 7 EWC Evaluation ↓ PHASE 8 External Memory ↓

PHASE 9 Adaptive Memory ↓ PHASE 10 Attention ↓ PHASE 11 Gated Fusion ↓ PHASE 12 Complete MAVIS
↓ PHASE 13 Tiny ImageNet Full Training ↓ PHASE 14 Ablations ↓ PHASE 15 Memory-Budget Study ↓
PHASE 16 Analysis ↓ PHASE 17 Graphs + Tables ↓ PHASE 18 Final Report ↓ PHASE 19 Presentation +
Demo

45. Definition of Done

The project is complete when all of the following are satisfied.

Infrastructure

- ☐ TensorFlow GPU verified.
- ☐ Environment recorded.
- ☐ Tiny ImageNet pipeline working.
- ☐ Continual task generator working.
- ☐ Checkpoint system working.

Models

- ☐ CNN baseline implemented.
- ☐ EWC implemented.
- ☐ External memory implemented.
- ☐ Attention retrieval implemented.
- ☐ Gated fusion implemented.
- ☐ MAVIS integrated.

Experiments

- ☐ CNN trained sequentially.
- ☐ CNN+EWC trained.
- ☐ CNN+Attention Memory trained.
- ☐ MAVIS trained.
- ☐ Memory-budget experiments completed.
- ☐ Ablations completed.

Evaluation

- ☐ Task accuracy matrix generated.
- ☐ Average accuracy calculated.
- ☐ Final accuracy calculated.
- ☐ Forgetting calculated.
- ☐ Old/new class accuracy calculated.
- ☐ Memory usage recorded.

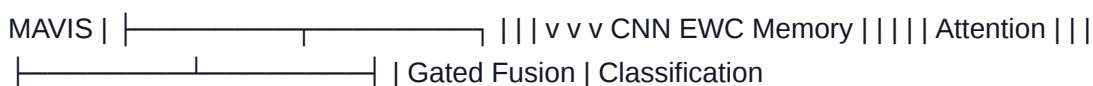
- ☐ Training time recorded.
- ☐ Parameter count recorded.
- ☐ GPU memory recorded.

Research output

- ☐ Accuracy plots generated.
- ☐ Forgetting plots generated.
- ☐ Memory-budget plots generated.
- ☐ Ablation plots generated.
- ☐ Final comparison table generated.
- ☐ Failure analysis completed.
- ☐ Limitations documented.
- ☐ Reproducibility information documented.

46. Final Technical Summary

MAVIS is a continual-learning architecture combining three ideas:



The CNN learns visual representations. EWC protects important parameters from destructive changes. External memory stores selected old visual representations. Attention selectively retrieves relevant memories. Adaptive memory selection maintains a fixed memory budget. Gated fusion combines current and retrieved knowledge.

The system is evaluated not only by classification accuracy, but also by forgetting, memory usage, computation, and scalability.

The central experimental comparison is: CNN -> CNN+EWC -> CNN+AttentionMemory -> MAVIS

The project therefore becomes a controlled investigation of:

parameter protection + representation memory + selective retrieval under constrained memory.

47. Recommended Initial Hyperparameters

These are starting values for implementation, not final optimized values.

Dataset: Tiny ImageNet 200 classes 10 tasks 20 classes/task

Input: 64x64x3 Batch: 64 Feature dimension: 512 Fusion dimension: 512

Attention: 4 heads key/query dimension ≈ 256

Memory: 500 entries initially

EWC: $\lambda = 1000$ initially Fisher samples = 1000/task

Optimizer: Adam Initial learning rate: $1e-3$ Epochs: 20/task for initial full experiments Precision: mixed precision if stable Seed: fixed and recorded

The first goal is not to optimize these values. The first goal is to establish a stable, reproducible baseline and then tune only the parameters justified by experiments.

48. Immediate Implementation Sequence

The next coding session should implement only:

1. Create MAVIS project directory
2. Create Python virtual environment
3. Install TensorFlow
4. Verify RTX 4050
5. Download/prepare Tiny ImageNet
6. Implement dataset loader
7. Implement 10-task generator
8. Visualize several samples from each task
9. Implement CNN encoder
10. Train CNN on a tiny subset
11. Train CNN on Task 1
12. Add continual evaluation
13. Train Tasks 1-2
14. Verify catastrophic forgetting
15. Only then implement EWC

This order minimizes debugging complexity and ensures every research component has a verified foundation.